Algoritmos de Apoio à Tomada de Decisão

Classificação de produtos com desconto: comparação de algoritmos de aprendizagem automática segundo o CRISP-DM

Trabalho prático

[Nome(s) e número(s) de estudante]

Docente: Bruno Veloso

Faculdade de Economia da Universidade do Porto (FEP)

Ano letivo 2026/2027

Outubro de 2026

Índice

Resumo / Abstract **3**
1. Introdução **4**
2. Metodologia **5**
2.1 Compreensão do negócio (Business Understanding) 5
2.2 Compreensão dos dados (Data Understanding) 5
2.3 Preparação dos dados (Data Preparation) 14
2.4 Modelação (Modeling) 19
2.5 Avaliação (Evaluation) 19
3. Resultados **21**
3.1 Comparação dos modelos 21
3.2 Análise por modelo 22
3.3 Importância das variáveis 24
3.4 Verificações de robustez 26
3.5 Modelo final e submissão 27
4. Conclusão **28**
Referências **28**
Anexo A – Hiperparâmetros e grelhas de pesquisa **30**
Anexo B – Estrutura do código e reprodutibilidade **30**

Resumo / Abstract

Resumo. Este trabalho aborda um problema de classificação binária em que se pretende prever se um produto, vendido em diversos estabelecimentos, se encontra com desconto (y = 1) ou sem desconto (y = 0), a partir de 17 variáveis discretas (d) e 9 variáveis contínuas (x). Seguiu-se a metodologia CRISP-DM. A análise exploratória dos 3 676 registos de treino revelou um desequilíbrio moderado de classes (20,6% de produtos com desconto), distribuições assimétricas em várias variáveis contínuas, 49 valores em falta em x9 – todos associados a y = 0 – e 170 registos duplicados com etiqueta consistente. Na preparação dos dados trataram-se os valores em falta (mediana e indicador de ausência), avaliaram-se os valores atípicos, o desequilíbrio (sobreamostragem aleatória, SMOTE, subamostragem e ponderação de classes) e a redundância entre variáveis, e o conjunto de treino foi dividido em treino interno (80%) e validação (20%), com validação cruzada estratificada de 5 folds no treino interno. Foram afinados e comparados nove algoritmos – KNN, árvore de decisão, Naive Bayes, MLP, Random Forest, Bagging, AdaBoost, Gradient Boosting e Extra Trees – através de accuracy, precision, recall e F-measure. Os métodos baseados em conjuntos de árvores foram claramente superiores: obtiveram entre 94,6% e 95,9% de accuracy na validação, contra 88,7% do KNN, 78,9% do MLP e 53,3% do Naive Bayes. O modelo selecionado por validação cruzada (AdaBoost, accuracy de 95,1% em CV) foi reajustado com todos os dados de treino e utilizado para gerar o ficheiro de submissão. As diferenças entre os cinco melhores modelos são, no entanto, inferiores ao erro-padrão das estimativas, pelo que não se pode afirmar que um deles seja superior aos restantes.

Palavras-chave: classificação, CRISP-DM, desequilíbrio de classes, ensembles, boosting, bagging, SMOTE.

Abstract. This work addresses a binary classification problem: predicting whether a product sold in several establishments is on sale (y = 1) or not (y = 0), using 17 discrete (d) and 9 continuous (x) variables. The CRISP-DM process was followed. Exploratory analysis of the 3,676 training records showed a moderate class imbalance (20.6% positives), skewed continuous variables, 49 missing values in x9 – all of them with y = 0 – and 170 duplicated records with consistent labels. Data preparation covered missing values (median imputation plus a missingness indicator), outliers, class imbalance (random oversampling, SMOTE, undersampling and class weights) and feature redundancy, and the training data were split into an inner training set (80%) and a hold-out validation set (20%), with stratified 5-fold cross-validation on the former. Nine algorithms – KNN, decision tree, Naive Bayes, MLP, Random Forest, Bagging, AdaBoost, Gradient Boosting and Extra Trees – were tuned and compared using accuracy, precision, recall and F-measure. Tree ensembles clearly outperformed the other families, reaching 94.6%–95.9% validation accuracy versus 88.7% for KNN, 78.9% for the MLP and 53.3% for Naive Bayes. The model selected by cross-validation (AdaBoost, 95.1% CV accuracy) was refitted on the full training set to produce the submission file. Differences among the five best models are smaller than the standard error of the estimates, so no single one can be declared superior.

Keywords: classification, CRISP-DM, class imbalance, ensembles, boosting, bagging, SMOTE.

1. Introdução

A decisão de aplicar um desconto a um produto é um problema recorrente do retalho: depende das características do produto, do estabelecimento e do contexto de venda, e é muitas vezes difícil de explicitar numa regra simples. Um modelo que consiga reconhecer, a partir dessas características, se um produto se encontra em promoção é útil para auditar políticas comerciais, detetar inconsistências nos dados publicados e apoiar a monitorização de preços. Este trabalho, desenvolvido no âmbito da unidade curricular de Algoritmos de Apoio à Tomada de Decisão, tem como objetivo construir e comparar classificadores capazes de prever essa condição.

1.1 Descrição do problema

Os dados publicados contêm informação sobre um conjunto de produtos em diferentes estabelecimentos, que ocasionalmente são vendidos com desconto. A variável a prever é y (y = 1 significa «com desconto»); todas as restantes podem ser usadas como preditoras, com exceção de i, o número do registo, que deve ser ignorado. As variáveis com prefixo d são discretas (aqui, todas binárias) e as variáveis com prefixo x são contínuas (algumas assumem poucos valores inteiros). O enunciado fornece dois ficheiros: **training.csv**, com 3 676 registos e a variável y conhecida, e **task.csv**, com 3 676 registos em que y é desconhecido e tem de ser previsto. A métrica de avaliação da competição é a accuracy e a submissão deve ser um ficheiro com as colunas i e y.

1.2 Objetivos e contributos

- Realizar uma análise exploratória completa (estatística descritiva, histogramas, diagramas de caixa, correlações e testes de associação com o alvo).
- Preparar os dados de forma rigorosa: valores em falta, valores atípicos, desequilíbrio de classes, redundância de variáveis e criação de dados de validação cruzada, garantindo que nenhum passo usa informação do conjunto de validação.
- Treinar, afinar e comparar KNN, árvores de decisão, Naive Bayes, redes neuronais (MLP), Random Forest, Bagging e Boosting, com uma tabela comparativa de accuracy, precision, recall e F-measure.
- Selecionar o modelo final com base em validação cruzada, avaliar a sua robustez e produzir as previsões para o ficheiro task.csv.

1.3 Estrutura do documento

O capítulo 2 descreve a metodologia segundo as seis fases do CRISP-DM (Chapman et al., 2000) – as cinco primeiras ocupam as subsecções 2.1 a 2.5 e a fase de implementação (deployment) é discutida no capítulo 4. O capítulo 3 apresenta os resultados, o capítulo 4 as conclusões, limitações e trabalho futuro. Os anexos reúnem os hiperparâmetros e a descrição do código, que permite reproduzir todos os resultados.

2. Metodologia

A metodologia segue o CRISP-DM (Cross-Industry Standard Process for Data Mining), um modelo de processo iterativo composto por seis fases – compreensão do negócio, compreensão dos dados, preparação dos dados, modelação, avaliação e implementação – em que as lições de cada fase podem obrigar a regressar às anteriores. Todo o trabalho foi implementado em Python 3 com as bibliotecas pandas, scikit-learn (Pedregosa et al., 2011), imbalanced-learn (Lemaître et al., 2017), SciPy, statsmodels, matplotlib e seaborn; a semente aleatória foi fixada em 42.

2.1 Compreensão do negócio (Business Understanding)

Objetivo de negócio. Identificar corretamente os produtos que se encontram com desconto a partir das suas características, de forma a automatizar uma tarefa que, de outro modo, exigiria inspeção manual.

Critério de sucesso. A métrica oficial é a accuracy, isto é, a proporção de produtos classificados corretamente. Como a classe positiva representa apenas cerca de um quinto dos casos, um classificador trivial que preveja sempre «sem desconto» obteria cerca de 79,4% de accuracy; qualquer modelo útil tem de superar claramente este valor de referência (baseline). Dado que a accuracy pode esconder um fraco desempenho na classe minoritária, reportam-se também a precision, o recall e a F-measure da classe positiva:

- **Precision** = VP / (VP + FP): dos produtos classificados como «com desconto», que fração o é realmente;
- **Recall** = VP / (VP + FN): dos produtos realmente com desconto, que fração é identificada;
- **F-measure (F1)** = 2·Precision·Recall / (Precision + Recall): média harmónica das duas anteriores.

Situação, recursos e restrições. Dispõe-se de dois ficheiros de dados e de nenhuma informação adicional sobre o significado concreto das variáveis (apenas o seu tipo). Não há dados sobre custos de erro assimétricos, pelo que se assume que um falso positivo e um falso negativo têm o mesmo custo, tal como implica a accuracy. A análise é, por isso, puramente baseada em dados, e as interpretações do papel de cada variável devem ser entendidas como hipóteses.

Objetivo de modelação. Aprender uma função f(d1,…,d18, x1,…,x9) → {0, 1} que maximize a accuracy em dados não vistos e produzir as previsões para os 3 676 registos de task.csv.

2.2 Compreensão dos dados (Data Understanding)

2.2.1 Descrição geral

O ficheiro de treino tem 3 676 linhas e 28 colunas: o identificador i, o alvo y, 17 variáveis discretas (d1–d5, d7–d18; não existe d6) e 9 variáveis contínuas (x1–x9). O ficheiro task.csv tem a mesma estrutura, com y em branco. O identificador i varia entre 2 e 7 352 no treino e os dois ficheiros são disjuntos em i (os registos foram repartidos entre treino e teste), pelo que i funciona apenas como identificador; a sua correlação com y é desprezável (−0,023) e, em conformidade com o enunciado, não foi utilizado.

A variável alvo está moderadamente desequilibrada: 2 920 registos (79,4%) sem desconto e 756 (20,6%) com desconto, o que equivale a um rácio de cerca de 3,9 : 1 (Figura 1).

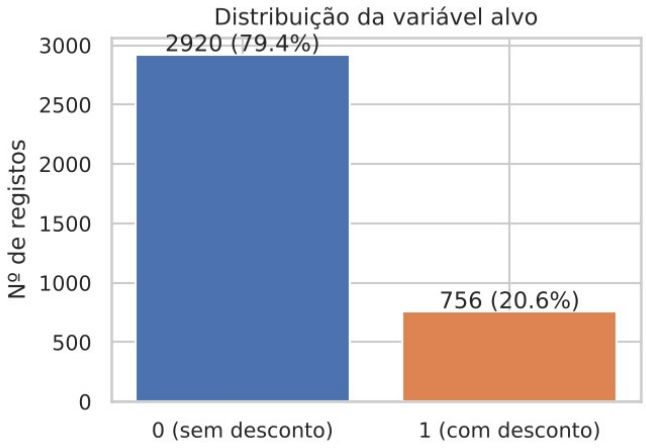


Figura 1 – Distribuição da variável alvo no conjunto de treino.

Quanto à qualidade dos dados, a única variável com valores em falta é x9: 49 no treino (1,33%) e 30 em task.csv. Nas restantes variáveis não se detetaram valores impossíveis (os intervalos observados são plausíveis). Existem 170 registos de treino com o vetor de características idêntico a outro registo (4,6%); em todos os grupos de duplicados a etiqueta é igual (não há contradições), o que sugere produtos genuinamente repetidos e não erros. Além disso, 382 registos de task.csv (10,4%) têm um vetor de características igual a algum registo de treino. Estas duplicações têm implicações para a avaliação dos modelos, discutidas na secção 3.4.

2.2.2 Variáveis contínuas

A Tabela 1 resume as variáveis contínuas. Verifica-se que x1 (valores inteiros de 1 a 7), x2 (1 a 4), x4 (1 a 3), x6, x7 e x8 assumem poucos valores distintos, embora tenham sido fornecidas como contínuas; x3 e x5 têm muitos valores distintos. x3 e x5 têm assimetria fortemente positiva (3,8 e 3,6) e curtose elevada (28,5 e 17,6), isto é, a maioria dos valores é pequena e há uma cauda longa de valores grandes. Os valores de x5 (0,39 a 27,48) terminam frequentemente em .49, .69 ou .99, o que sugere – sem que tal esteja confirmado – que representa um preço. x4 é praticamente constante: 89,7% dos registos valem 1, 10,0% valem 2 e apenas 11 registos valem 3.

Tabela 1 – Estatística descritiva das variáveis contínuas (conjunto de treino).

var.	n	média	dp	mín	Q1	mediana	Q3	máx	assim.	curtose	n.º val.	falta
x1	3676	4,03	2,00	1,00	2,00	4,00	6,00	7,00	-0,03	-1.251	7	0
x2	3676	2,05	0,98	1,00	1,00	2,00	3,00	4,00	0,47	-0.894	4	0
x3	3676	0,55	0,39	0,00	0,35	0,50	0,75	5,00	3,81	28.484	72	0
x4	3676	1,11	0,32	1,00	1,00	1,00	1,00	3,00	2,85	7.17	3	0
x5	3676	2,58	2,78	0,39	1,09	1,69	2,82	27,48	3,58	17.613	181	0
x6	3676	5,99	4,77	1,00	1,00	5,00	8,00	17,00	0,73	-0.56	15	0
x7	3676	8,89	7,40	1,00	4,00	6,00	12,00	25,00	1,05	-0.205	25	0
x8	3676	4,31	1,62	1,00	3,00	4,00	5,00	12,00	0,66	0.625	12	0
x9	3627	26,32	15,52	1,00	13,50	28,00	40,00	54,00	-0,13	-1.221	54	49

A Figura 2 mostra os histogramas. x1 e x9 são aproximadamente uniformes; x2, x6, x7 e x8 são discretas e ligeiramente assimétricas à direita; x3, x4 e x5 concentram-se à esquerda com caudas longas. Os diagramas de caixa (Figura 3) confirmam a presença de valores extremos em x3, x5, x7 e x8, e a degenerescência de x4, em que a caixa colapsa num único valor.

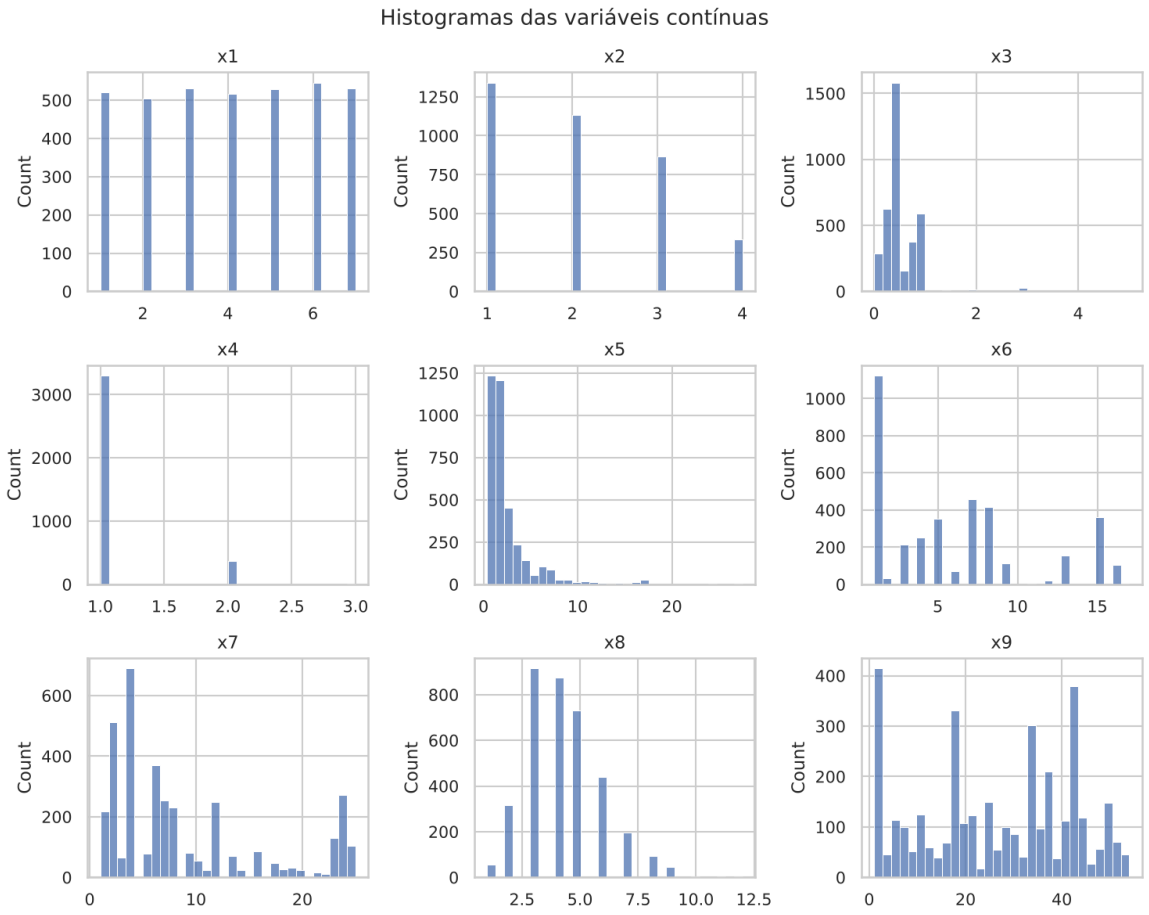


Figura 2 – Histogramas das variáveis contínuas.

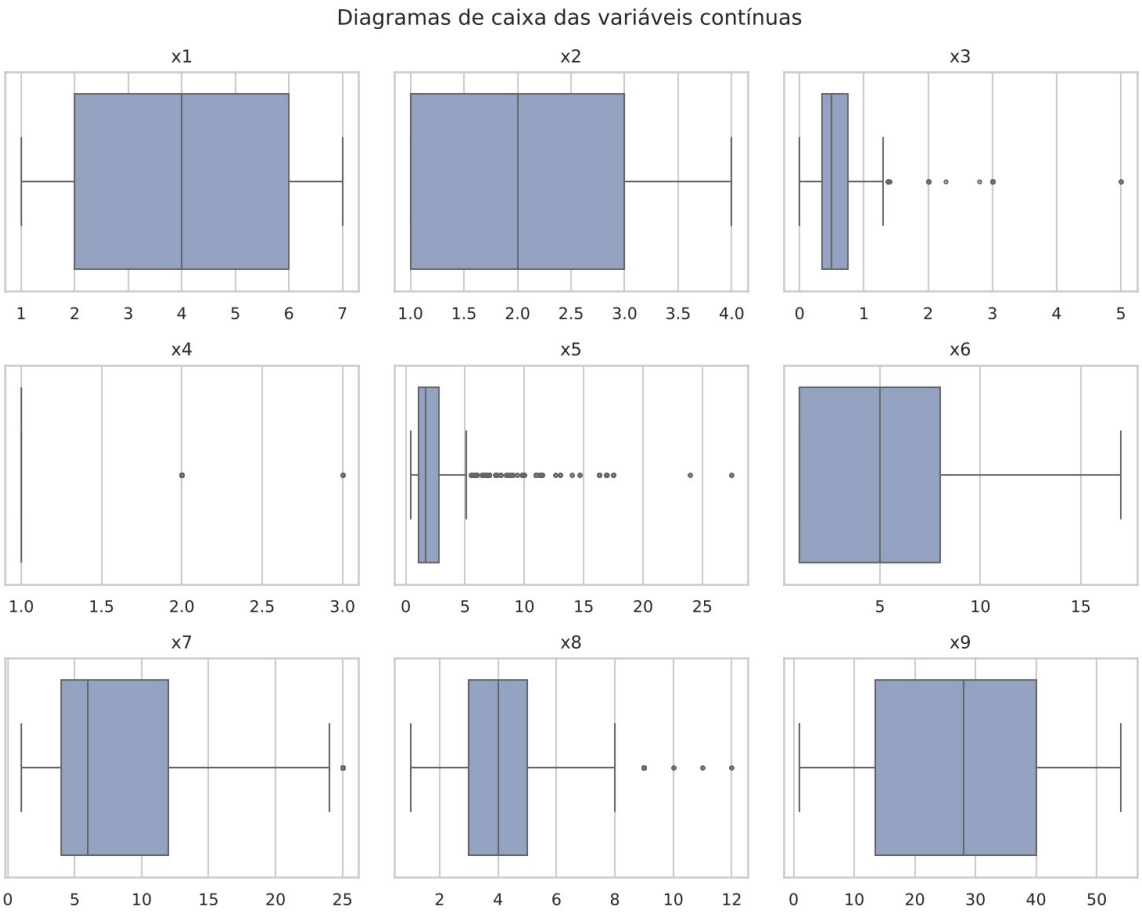


Figura 3 – Diagramas de caixa das variáveis contínuas.

As densidades por classe (Figura 4) e os diagramas de caixa por classe (Figura 5) permitem uma primeira leitura do poder discriminativo. As distribuições de x5 e x9 diferem visivelmente entre classes: os produtos com desconto tendem a ter valores de x5 mais elevados e valores de x9 mais altos, e não aparecem com x9 em falta. Para x1, x2 e x8 as distribuições das duas classes quase coincidem.

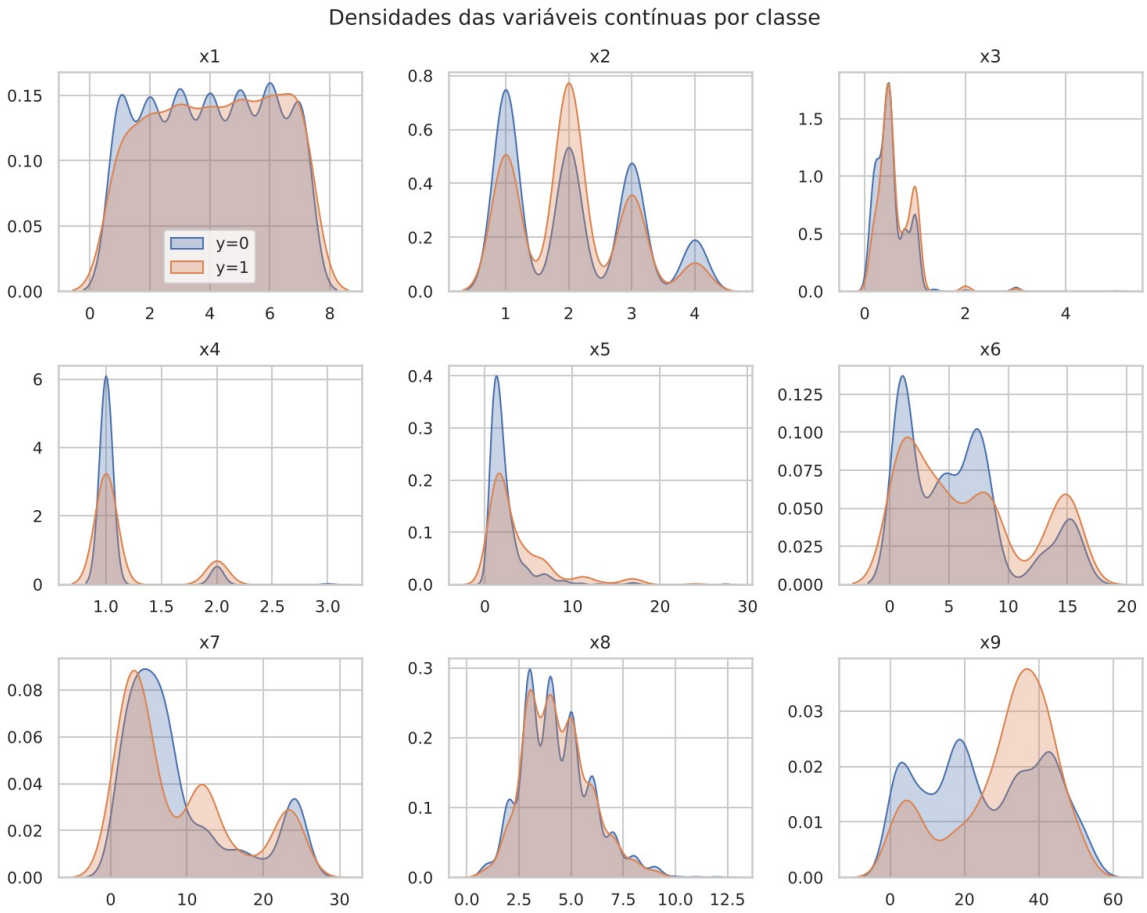


Figura 4 – Densidades das variáveis contínuas por classe.

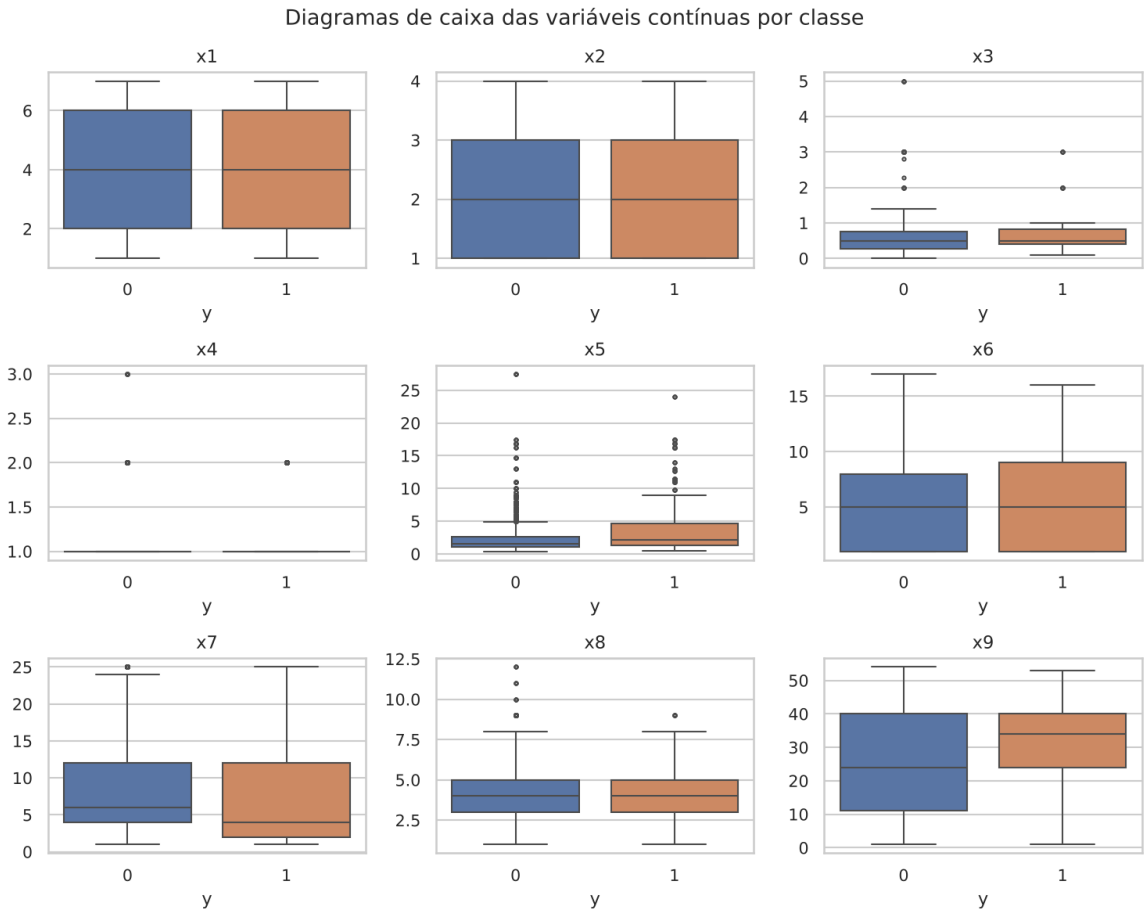


Figura 5 – Diagramas de caixa das variáveis contínuas por classe.

2.2.3 Variáveis discretas

As 17 variáveis d são binárias e a soma por linha varia entre 0 e 3 (371 registos com soma 0, 2 775 com soma 1, 521 com soma 2 e 9 com soma 3). O comportamento é compatível com uma codificação one-hot incompleta (uma variável por categoria) de uma ou mais variáveis categóricas, mas o enunciado não o confirma. A Tabela 2 mostra a prevalência de cada variável e a taxa de desconto condicional; a Figura 6 representa os mesmos dados.

Tabela 2 – Prevalência das variáveis discretas e taxa de desconto condicional (treino; taxa global = 20,6%).

var.	prevalência (%)	n (d=1)	P(y=1 | d=1) (%)	P(y=1 | d=0) (%)
d1	5,93	218	30,28	19,95
d2	13,96	513	24,76	19,89
d3	2,01	74	21,62	20,54
d4	17,06	627	25,04	19,65
d5	11,34	417	3,12	22,80
d7	1,52	56	23,21	20,52
d8	2,04	75	33,33	20,30
d9	12,84	472	11,02	21,97
d10	2,67	98	9,18	20,88
d11	6,09	224	50,00	18,66
d12	2,31	85	27,06	20,41
d13	6,75	248	27,02	20,10
d14	5,06	186	16,13	20,80
d15	2,80	103	0,97	21,13
d16	2,83	104	19,23	20,60

d17	2,80	103	31,07	20,26
d18	6,56	241	11,62	21,19

Algumas categorias têm um efeito muito forte sobre a probabilidade de desconto: com d11 = 1, 50,0% dos produtos estão em desconto (contra 18,7% sem d11); com d5 = 1 apenas 3,1% (contra 22,8%), com d15 = 1 apenas 1,0% (contra 21,1%) e com d9 = 1, 11,0%. Outras, como d3, d7 e d16, estão praticamente alinhadas com a taxa global e, além disso, são raras (prevalência de 2,0%, 1,5% e 2,8%).

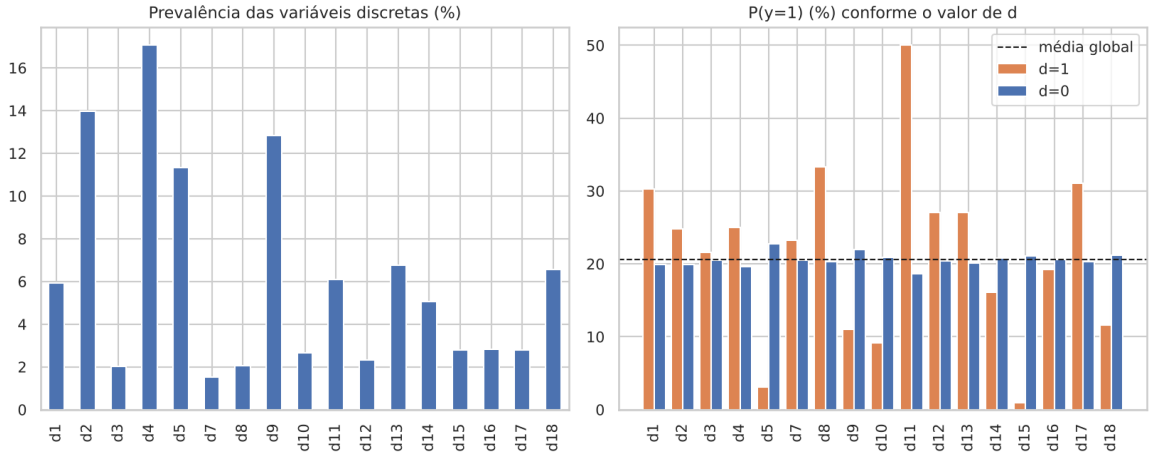


Figura 6 – Prevalência das variáveis discretas (esquerda) e taxa de desconto condicional (direita).

2.2.4 Correlações e testes de associação com o alvo

A Figura 7 mostra a matriz de correlação de todas as variáveis (Pearson; entre variáveis binárias e y equivale ao coeficiente ponto-bisserial/phi). As correlações lineares com y são todas fracas: a mais forte é a de x5 (0,216), seguida de d11 (0,186), d5 (−0,154), x9 (0,125) e x4 (0,113). Entre as variáveis contínuas as maiores correlações são x3–x5 (0,53) e x3–x4 (0,48) (Figura 8). Entre pares que envolvem variáveis discretas, os valores mais altos são d1–x4 (0,71) e d10–d13 (0,62).

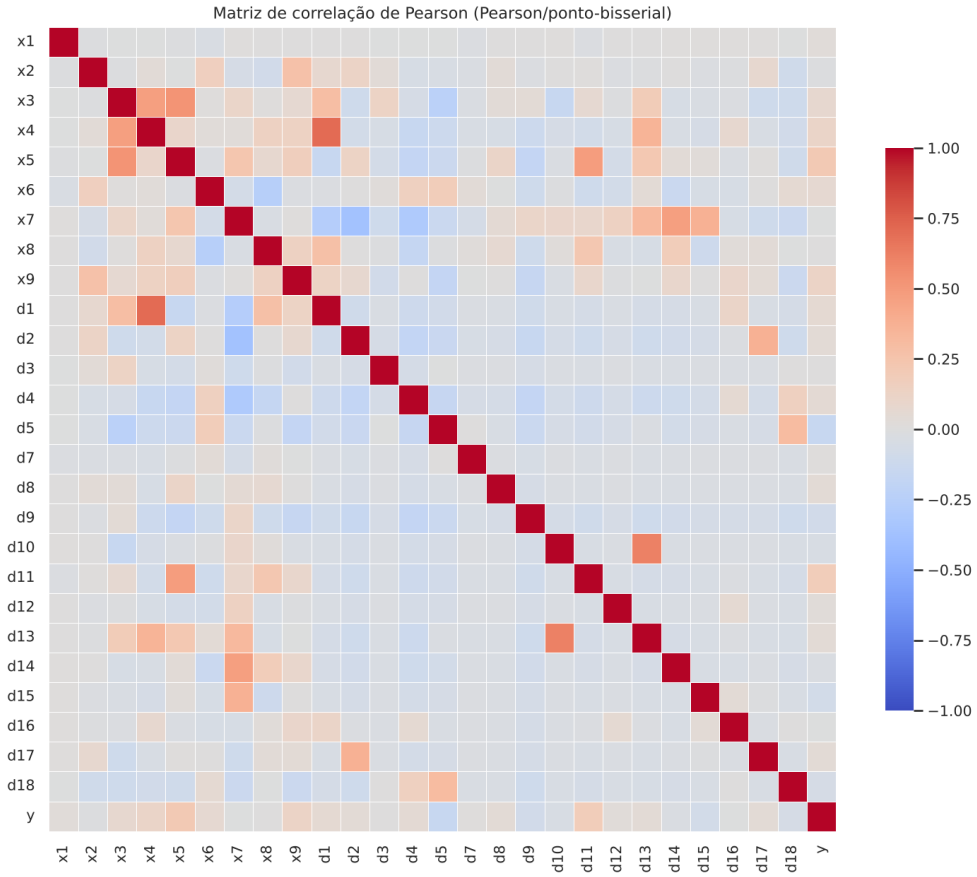


Figura 7 – Matriz de correlação de todas as variáveis, incluindo y.

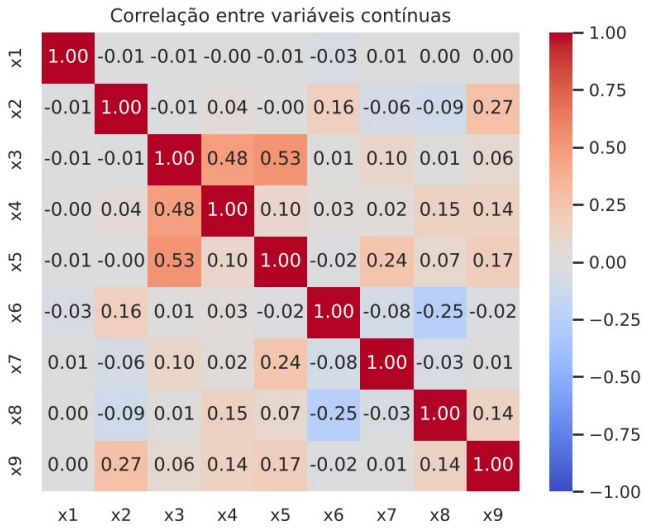


Figura 8 – Correlação entre as variáveis contínuas.

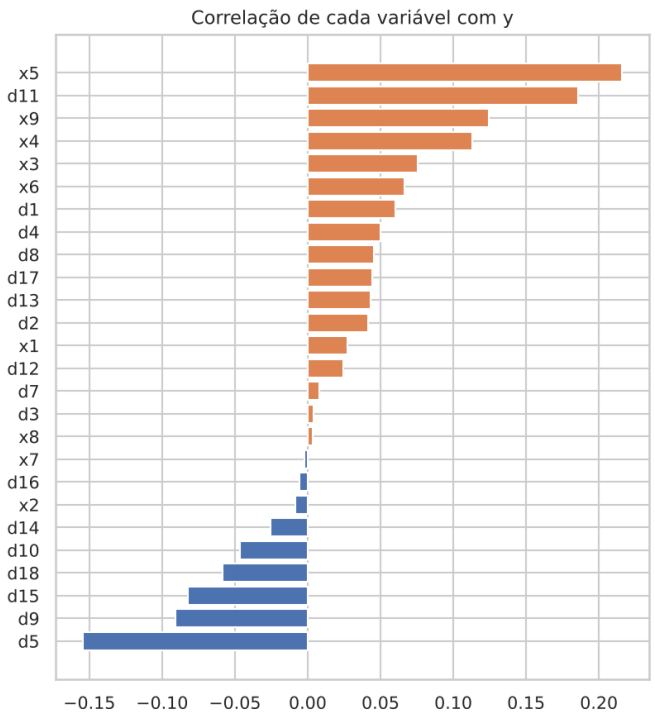


Figura 9 – Correlação de cada variável com o alvo.

Como as correlações lineares são baixas, mas o conjunto de variáveis pode ainda ter relação não linear com y, aplicaram-se testes de hipóteses: o teste de Mann-Whitney para as variáveis contínuas (não assume normalidade, o que seria inadequado dadas as assimetrias observadas) e o teste do qui-quadrado de independência para as discretas. Com um nível de significância de 5%, 18 das 26 variáveis apresentam associação estatisticamente significativa com y: x3, x4, x5, x6, x7, x9 e d1, d2, d4, d5, d8, d9, d10, d11, d13, d15, d17, d18. Não se rejeita a independência para x1, x2, x8, d3, d7, d12, d14 e d16. Dado o número de testes (26), alguns resultados marginais (p entre 0,01 e 0,05, como x7, d2 ou d13) devem ser lidos com cautela, e esta análise univariada não capta interações – que é, como se verá, onde os modelos de árvores obtêm a maior parte do ganho.

Tabela 3 – Testes de associação entre cada variável e y.

variável	teste	valor-p	signif. a 5%	variável	teste	valor-p	signif. a 5%
x1	Mann-Whitney	0,0985	não	d5	Qui-quadrado	< 0,0001	sim
x2	Mann-Whitney	0,6914	não	d7	Qui-quadrado	0,7433	não
x3	Mann-Whitney	< 0,0001	sim	d8	Qui-quadrado	0,0088	sim
x4	Mann-Whitney	< 0,0001	sim	d9	Qui-quadrado	< 0,0001	sim
x5	Mann-Whitney	< 0,0001	sim	d10	Qui-quadrado	0,0070	sim
x6	Mann-Whitney	0,0008	sim	d11	Qui-quadrado	< 0,0001	sim
x7	Mann-Whitney	0,0111	sim	d12	Qui-quadrado	0,1730	não
x8	Mann-Whitney	0,4974	não	d13	Qui-quadrado	0,0117	sim
x9	Mann-Whitney	< 0,0001	sim	d14	Qui-quadrado	0,1489	não
d1	Qui-quadrado	0,0004	sim	d15	Qui-quadrado	< 0,0001	sim
d2	Qui-quadrado	0,0134	sim	d16	Qui-quadrado	0,8269	não
d3	Qui-quadrado	0,9349	não	d17	Qui-quadrado	0,0107	sim

d4	Qui-quadrado	0,0028	sim	d18	Qui-quadrado	0,0005	sim

2.3 Preparação dos dados (Data Preparation)

Todas as decisões de preparação foram tomadas apenas com dados de treino e implementadas dentro de pipelines (imputação → padronização → reamostragem → modelo), de modo que cada passo é ajustado exclusivamente com os dados de treino de cada fold. Isto evita fuga de informação (data leakage) entre treino e validação.

2.3.1 Divisão dos dados de treino

Para estimar o desempenho de forma honesta, o ficheiro training.csv foi dividido de forma estratificada por y em **treino interno (80%: 2 940 registos)** e **validação hold-out (20%: 736 registos)**. A proporção de y = 1 é de 20,58% no treino interno e 20,52% na validação, ou seja, a estratificação preservou a distribuição da classe. O protocolo é o seguinte: (i) a afinação de hiperparâmetros e a seleção do modelo fazem-se exclusivamente por **validação cruzada estratificada de 5 folds** no treino interno (2 352 registos de treino e 588 de teste por fold, com 20,58% de positivos em todos os folds); (ii) o conjunto de validação só é usado uma vez por modelo, para medir o desempenho em dados nunca vistos; (iii) o modelo selecionado é finalmente reajustado com os 3 676 registos de training.csv para prever task.csv.

2.3.2 Valores em falta

x9 tem 49 valores em falta (1,33%) no treino e 30 em task.csv. A análise revelou um facto relevante: **todos os 49 registos com x9 em falta têm y = 0**, enquanto entre os registos com x9 observado a taxa de y = 1 é de 20,8%. Os valores em falta não são, portanto, completamente aleatórios (não são MCAR): a ausência é informativa. Por esse motivo, para além de imputar, acrescenta-se uma variável indicadora de ausência. Compararam-se três imputações – mediana, média e KNN (k = 5) – com uma árvore de decisão e uma Random Forest (Tabela 4). As diferenças são desprezáveis (a accuracy da Random Forest varia entre 0,937 e 0,939), pelo que se adotou a imputação pela mediana, mais robusta a assimetrias e mais simples, complementada pelo indicador. O indicador aumentou a accuracy da Random Forest de 0,9367 para 0,9381 e a F-measure de 0,840 para 0,845, um ganho pequeno mas coerente com a evidência anterior.

Tabela 4 – Efeito do método de imputação (CV 5 folds no treino interno).

imputação | modelo	accuracy	F1
mediana|DT	0,8558	0,5560
mediana|RF	0,9367	0,8402
média|DT	0,8534	0,5539
média|RF	0,9388	0,8453
KNN (k=5)|DT	0,8480	0,5584
KNN (k=5)|RF	0,9391	0,8465

2.3.3 Valores atípicos (outliers)

Identificaram-se outliers pelo critério de Tukey (valores fora de [Q1 − 1,5·IQR; Q3 + 1,5·IQR]) e, em complemento, por |z| > 3 (Tabela 5). x3 tem 61 outliers (1,7%), incluindo um valor extremo de 5,0 (z ≈ 11); x5 tem 374 (10,2%) devido à cauda longa; x7 tem 103 (2,8%) e x8 55 (1,5%). x1, x2, x6 e x9 não

têm outliers. O caso de x4 merece nota: como o IQR é zero (Q1 = Q3 = 1), qualquer valor diferente de 1 é assinalado como outlier (377 registos, 10,3%), embora os valores 2 e 3 sejam categorias legítimas – o critério de Tukey não é aplicável a variáveis quase constantes.

Tabela 5 – Valores atípicos por variável contínua.

var.	limite inf.	limite sup.	n.º outliers (IQR)	%	n.º |z|>3
x1	-4,00	12,00	0	0,00	0
x2	-2,00	6,00	0	0,00	0
x3	-0,25	1,35	61	1,66	50
x4	1,00	1,00	377	10,26	11
x5	-1,50	5,42	374	10,17	102
x6	-9,50	18,50	0	0,00	0
x7	-8,00	24,00	103	2,80	0
x8	0,00	8,00	55	1,50	10
x9	-26,25	79,75	0	0,00	0

Decidiu-se **não remover nem alterar os valores atípicos**, por três razões: (i) são valores plausíveis (por exemplo, valores elevados de x5), não erros de recolha; (ii) os algoritmos baseados em árvores, que são os mais fortes neste problema, são invariantes a transformações monótonas e insensíveis a valores extremos; (iii) testou-se a winsorização às barreiras de Tukey (Figura 10) nos dois modelos mais sensíveis a escala e a distribuição, KNN e Naive Bayes, e esta não trouxe melhoria (Tabela 6: a accuracy do KNN passou de 0,8068 para 0,8051 e a do Naive Bayes de 0,5592 para 0,5524). A winsorização foi, assim, descartada. Os modelos sensíveis à escala (KNN, Naive Bayes e MLP) recebem, no entanto, variáveis padronizadas (média 0, desvio-padrão 1).

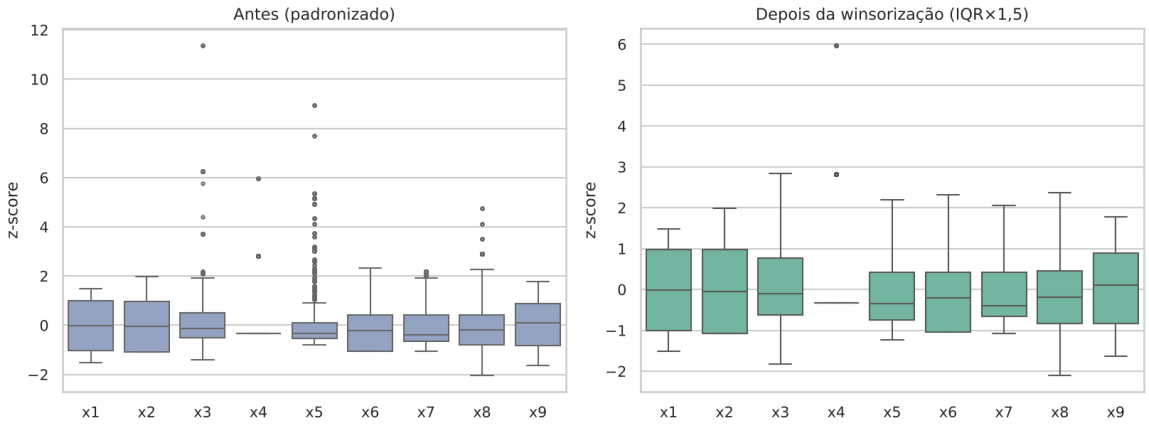


Figura 10 – Variáveis contínuas padronizadas antes e depois da winsorização (IQR × 1,5); x4 não é alterada por ter IQR = 0.

Tabela 6 – Efeito da winsorização (CV 5 folds no treino interno).

modelo | tratamento	accuracy	F1
KNN|sem winsorização	0,8068	0,3807
KNN|com winsorização	0,8051	0,3786
NB|sem winsorização	0,5592	0,4248
NB|com winsorização	0,5524	0,4248

2.3.4 Dados desequilibrados

Com 20,6% de positivos, o desequilíbrio é moderado, mas suficiente para que os classificadores favoreçam a classe maioritária. Compararam-se cinco estratégias, sempre aplicadas dentro de cada fold de treino (nunca no fold de teste): nenhum tratamento; ponderação das classes (class_weight = balanced); sobreamostragem aleatória da classe minoritária; SMOTE (Chawla et al., 2002), que gera exemplos sintéticos por interpolação entre vizinhos da classe minoritária; e subamostragem aleatória da classe maioritária. A comparação foi feita com árvore de decisão (profundidade 6), KNN (k = 15) e Random Forest (200 árvores).

Tabela 7 – Estratégias de tratamento do desequilíbrio (CV 5 folds no treino interno; métricas da classe 1).

estratégia	modelo	accuracy	precision	recall	F1
Sem tratamento	DT	0,856	0,756	0,446	0,556
Sem tratamento	KNN	0,807	0,558	0,289	0,381
Sem tratamento	RF	0,937	0,874	0,812	0,841
class_weight	DT	0,745	0,435	0,792	0,560
class_weight	RF	0,933	0,809	0,883	0,844
Sobreamostragem aleatória	DT	0,760	0,453	0,742	0,560
Sobreamostragem aleatória	KNN	0,689	0,374	0,759	0,501
Sobreamostragem aleatória	RF	0,941	0,853	0,865	0,858
SMOTE	DT	0,745	0,443	0,778	0,561
SMOTE	KNN	0,696	0,385	0,800	0,519
SMOTE	RF	0,942	0,861	0,856	0,858
Subamostragem aleatória	DT	0,692	0,395	0,830	0,530
Subamostragem aleatória	KNN	0,627	0,320	0,726	0,444
Subamostragem aleatória	RF	0,851	0,591	0,911	0,716

Os resultados (Tabela 7 e Figura 11) mostram que: (i) a subamostragem é claramente prejudicial, porque descarta informação (a accuracy da Random Forest cai de 0,937 para 0,851); (ii) para a árvore de decisão e o KNN, a reamostragem aumenta o recall à custa de uma forte quebra de precision e de accuracy (por exemplo, o KNN passa de 0,807 para 0,696 de accuracy com SMOTE), ainda que melhore a F-measure; (iii) para a Random Forest, a sobreamostragem aleatória e o SMOTE melhoram simultaneamente a accuracy (0,937 → 0,941/0,942) e a F-measure (0,841 → 0,858). Como o efeito depende do algoritmo, **a estratégia de reamostragem foi tratada como um hiperparâmetro**: na fase de modelação, cada algoritmo escolhe, por validação cruzada, entre «nenhuma» e SMOTE. Não se incluiu a subamostragem. A ponderação de classes não foi usada porque nem todos os algoritmos a suportam.

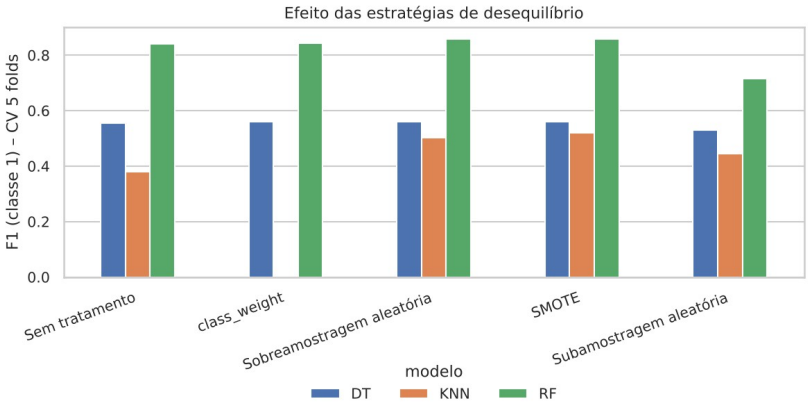


Figura 11 – Efeito das estratégias de tratamento do desequilíbrio na F-measure.

2.3.5 Redundância de variáveis

Analisou-se a redundância por quatro vias. **(1) Correlação:** os pares com maior correlação absoluta são d1–x4 (0,71), d10–d13 (0,62), x3–x5 (0,53), d11–x5 (0,48) e x3–x4 (0,48); nenhum ultrapassa 0,75, valor usual para sugerir a remoção. **(2) Multicolinearidade:** os fatores de inflação da variância (VIF) das variáveis contínuas são todos baixos (máximo 1,90, em x3; Tabela 9), muito abaixo do limiar habitual de 5–10. **(3) Variância quase nula:** d3, d7 e d8 têm variância inferior a 0,02 (prevalências de 2,0%, 1,5% e 2,0%). **(4) Informação mútua com y:** nove variáveis (d3, d16, d7, d12, d14, d2, d13, d17, d8) têm informação mútua inferior a 0,001, ou seja, praticamente nula (Figura 12); as mais informativas são x9 (0,120), x5 (0,118), x7 (0,061), x3 (0,060) e x6 (0,027).

Tabela 8 – Pares de variáveis com maior correlação absoluta.

variável 1	variável 2	|r|
d1	x4	0,709
d10	d13	0,615
x3	x5	0,526
d11	x5	0,480
x3	x4	0,476
d14	x7	0,471
d2	x7	0,375
d2	d17	0,369

Tabela 9 – Fatores de inflação da variância das variáveis contínuas.

	x1	x2	x3	x4	x5	x6	x7	x8	x9
VIF	1,00	1,13	1,90	1,43	1,59	1,11	1,08	1,15	1,16

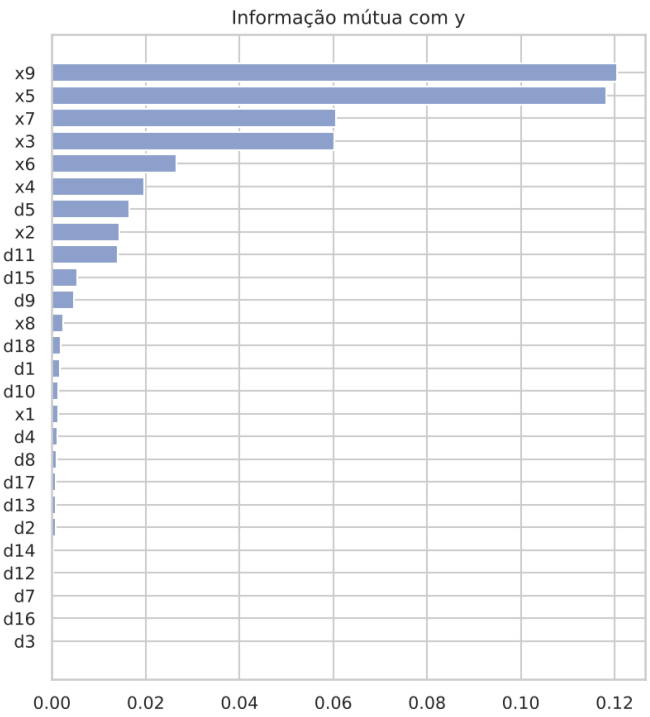


Figura 12 – Informação mútua entre cada variável e y.

Testou-se o efeito de remover variáveis com uma Random Forest em validação cruzada (Tabela 10). Remover x3 (a mais correlacionada com outras) ou x3 e x4 diminuiu a accuracy (0,9340 e 0,9316, contra 0,9367 com todas as variáveis). Remover as nove variáveis de informação mútua nula aumentou-a ligeiramente (0,9398), mas o ganho de 0,003 é inferior ao desvio-padrão entre folds (cerca de 0,008), pelo que não é estatisticamente distinguível de zero. Assim, **mantiveram-se todas as variáveis**: não existe redundância severa, os modelos de árvores lidam bem com variáveis pouco informativas e a informação mútua univariada pode subestimar variáveis que só são úteis em interação com outras (a variável d2, por exemplo, tem MI quase nula mas está correlacionada com x7).

Tabela 10 – Efeito da remoção de variáveis (Random Forest, CV 5 folds).

conjunto de variáveis	n.º variáveis	accuracy	F1
todas	26	0,9367	0,8402
sem x3 (corr. com x4/x5)	25	0,9340	0,8322
sem x3 e x4	24	0,9316	0,8247
sem variáveis |MI|≈0	17	0,9398	0,8472

2.3.6 Criação dos dados de validação cruzada

A validação cruzada foi construída com StratifiedKFold (5 folds, embaralhamento, semente 42) sobre o treino interno, de modo que cada fold mantém a proporção de 20,58% de positivos (588 registos de teste e 2 352 de treino por fold). Este mesmo objeto de validação cruzada é partilhado por todos os modelos, para que as comparações sejam feitas exatamente sobre as mesmas partições. Quando a reamostragem (SMOTE) é usada, é aplicada apenas ao conjunto de treino de cada fold, através de um pipeline do imbalanced-learn.

2.4 Modelação (Modeling)

2.4.1 Algoritmos

Compararam-se nove algoritmos de famílias distintas (Tabela 11). Todos partem do mesmo pré-processamento: imputação pela mediana com indicador de ausência de x9; padronização apenas nos modelos sensíveis à escala (KNN, Naive Bayes e MLP); e reamostragem opcional por SMOTE, escolhida por validação cruzada.

Tabela 11 – Algoritmos comparados.

algoritmo	ideia principal	hiperparâmetros afinados
KNN	Classifica pela maioria (ponderada pela distância) dos k vizinhos mais próximos (Cover & Hart, 1967)	n.º vizinhos, ponderação, métrica (L1/L2)
Árvore de Decisão	Partições recursivas por impureza de Gini ou entropia (CART; Breiman et al., 1984)	profundidade, folha mínima, critério
Naive Bayes	Regra de Bayes com independência condicional; Gaussiano	var_smoothing
MLP	Rede neuronal com camadas densas, treinada por retropropagação (Rumelhart et al., 1986)	camadas, regularização L2, taxa de aprendizagem
Random Forest	Bagging de árvores com subconjunto aleatório de variáveis (Breiman, 2001)	n.º árvores, max_features, folha mínima
Bagging	Árvores treinadas em amostras bootstrap, votação (Breiman, 1996)	n.º árvores, max_samples, profundidade
AdaBoost	Boosting sequencial com reponderação dos erros (Freund & Schapire, 1997)	n.º estimadores, taxa, profundidade base
Gradient Boosting	Boosting aditivo por gradiente de árvores pequenas (Friedman, 2001)	n.º árvores, taxa, profundidade, subamostra
Extra Trees	Árvores com cortes aleatórios, sem bootstrap (Geurts et al., 2006)	n.º árvores, max_features, folha mínima

O **Extra Trees** não estava na lista pedida mas foi incluído por ser uma variante de Random Forest muito competitiva em dados de dimensão reduzida; os resultados dos oito algoritmos pedidos podem ser lidos sem ele. Considerou-se o conjunto Random Forest, Bagging, AdaBoost e Gradient Boosting como as famílias de «bagging» e «boosting».

2.4.2 Afinação de hiperparâmetros e desenho do teste

A afinação fez-se com GridSearchCV (pesquisa exaustiva) sobre as grelhas do Anexo A, usando como critério a **accuracy média em validação cruzada de 5 folds** no treino interno (a métrica da competição). O Gradient Boosting, mais caro, foi afinado com RandomizedSearchCV (12 combinações). Em todos os casos a escolha entre «sem reamostragem» e SMOTE faz parte da grelha. Após a escolha da melhor combinação, o modelo é reajustado em todo o treino interno e avaliado (i) por validação cruzada, reportando média e desvio-padrão e (ii) no conjunto de validação hold-out, reportando accuracy, precision, recall, F-measure, AUC-ROC e precisão média (área sob a curva precision-recall).

2.5 Avaliação (Evaluation)

A avaliação segue três níveis. **Técnico:** accuracy, precision, recall e F-measure da classe «com desconto», matrizes de confusão, curvas ROC e curvas precision-recall. **Estatístico:** como o conjunto de validação tem 736 registos, o erro-padrão de uma accuracy de 0,95 é de cerca de 0,8 pontos

percentuais (√(0,95·0,05/736)), e o desvio-padrão da accuracy entre folds de CV é de cerca de 0,8 pontos; diferenças inferiores a este valor não devem ser interpretadas como superioridade. **De negócio:** comparação com a baseline (79,4%) e análise do tipo de erros (falsos positivos vs. falsos negativos).

3. Resultados

3.1 Comparação dos modelos

A Tabela 12 apresenta, para cada algoritmo com os hiperparâmetros selecionados por validação cruzada, a accuracy média (± desvio-padrão) em CV no treino interno e as quatro métricas no conjunto de validação. As colunas Precision, Recall e F-measure referem-se à classe «com desconto». A linha a negrito corresponde ao modelo selecionado pelo critério de maior accuracy em validação cruzada.

Tabela 12 – Tabela comparativa dos modelos (CV no treino interno; restantes métricas no conjunto de validação, classe 1). Baseline (maioria): accuracy = 0,794.

Modelo	CV accuracy	CV F1	Accuracy	Precision	Recall	F-measure	AUC-ROC
AdaBoost	**0,9510 ± 0,0084**	**0,879**	**0,946**	**0,905**	**0,821**	**0,861**	**0,957**
Gradient Boosting	0,9493 ± 0,0077	0,874	0,951	0,932	0,821	0,873	0,964
Extra Trees	0,9463 ± 0,0077	0,869	0,958	0,929	0,861	0,893	0,977
Random Forest	0,9429 ± 0,0084	0,861	0,950	0,913	0,834	0,872	0,971
Bagging	0,9415 ± 0,0081	0,858	0,959	0,942	0,854	0,896	0,959
Árvore de Decisão	0,9194 ± 0,0051	0,810	0,942	0,855	0,861	0,858	0,912
KNN	0,8704 ± 0,0205	0,652	0,887	0,793	0,609	0,689	0,906
MLP	0,8639 ± 0,0182	0,693	0,789	0,491	0,748	0,593	0,846
Naive Bayes	0,5486 ± 0,0265	0,424	0,533	0,278	0,801	0,413	0,710

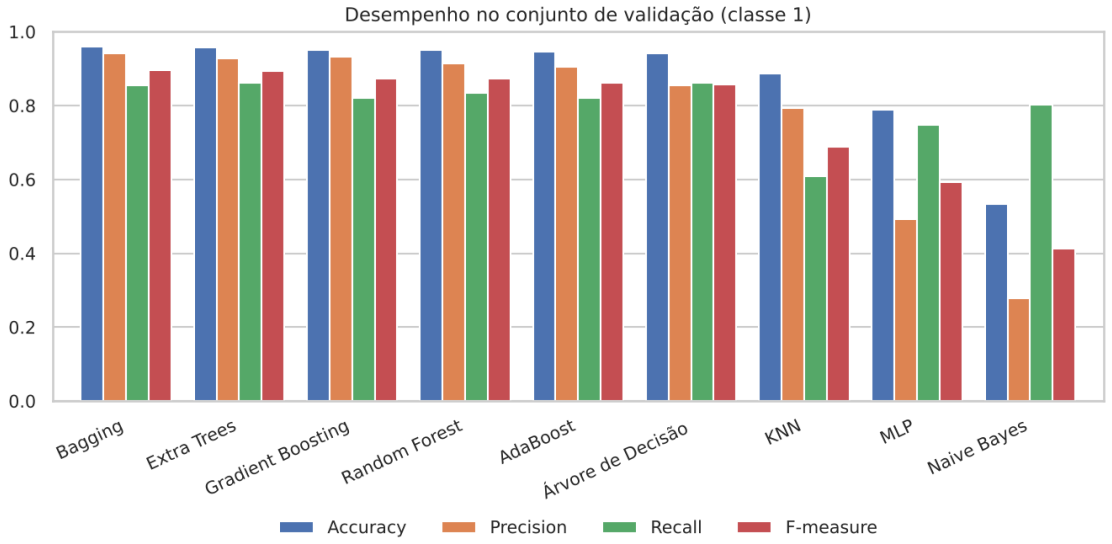


Figura 13 – Accuracy, precision, recall e F-measure dos modelos no conjunto de validação.

Observam-se três grupos de desempenho bem definidos:

- **Conjuntos de árvores (Bagging, Extra Trees, Gradient Boosting, Random Forest, AdaBoost):** accuracy na validação entre 0,946 e 0,959, F-measure entre 0,861 e 0,896, AUC entre 0,957 e 0,977. Apresentam precision elevada (0,905–0,942) e recall mais modesto (0,821–0,861), isto é, são conservadores: raramente classificam erradamente um produto sem desconto como em desconto, mas perdem entre 14% e 18% dos produtos realmente em desconto.
- **Árvore de decisão isolada:** accuracy de 0,942 na validação (0,919 em CV) e o recall mais equilibrado face à precision (0,861 e 0,855). Beneficia do SMOTE e, sem limite de profundidade, ajusta-se de forma muito fina aos dados de treino.
- **KNN, MLP e Naive Bayes:** claramente inferiores. O KNN (0,887) supera a baseline mas tem recall baixo (0,609); o MLP (0,789 na validação) fica abaixo da baseline na accuracy, apesar de recall de 0,748; o Naive Bayes (0,533) é pior do que classificar tudo como «sem desconto».

A diferença entre a accuracy do melhor e do quinto melhor conjunto de árvores na validação é de 1,3 pontos percentuais, comparável ao erro-padrão da estimativa (≈ 0,8 pontos); as diferenças em CV são ainda menores (0,95 vs. 0,94 com desvio-padrão ≈ 0,008). **Não há, portanto, evidência estatística de que um dos cinco conjuntos de árvores seja melhor do que os outros.** O que é estatisticamente claro é a superioridade deste grupo sobre KNN, MLP e Naive Bayes.

3.2 Análise por modelo

KNN

A melhor configuração usou k = 7, distância de Manhattan e ponderação pela distância, sem reamostragem. Atingiu 88,7% de accuracy na validação mas só 60,9% de recall (92 de 151 positivos). O KNN depende de uma noção de distância que, com 26 variáveis – 17 binárias e 9 contínuas em escalas diferentes – é pouco informativa, e sofre com a maldição da dimensionalidade. A reamostragem com SMOTE aumentou o recall mas reduziu a accuracy, e por isso não foi selecionada.

Árvore de decisão

A árvore ótima não tem limite de profundidade, usa o critério de Gini e foi treinada com SMOTE. Na validação obtém 0,942 de accuracy com 130 verdadeiros positivos e 21 falsos negativos – o menor número de falsos negativos, a par do Extra Trees – ao custo de 22 falsos positivos. É o modelo mais interpretável, mas tem, porém, accuracy em CV (0,919) 2,2 pontos abaixo da obtida na validação (0,942), uma diferença maior do que nos conjuntos de árvores, que pode refletir a presença, na validação, de registos idênticos a registos de treino.

Naive Bayes

O Naive Bayes Gaussiano é inadequado a estes dados: assume independência condicional entre variáveis e distribuições normais, ambas violadas (variáveis binárias, assimetrias fortes, correlações entre variáveis). Mesmo com a maior suavização testada (var_smoothing = 0,1), classifica incorretamente 314 dos 585 produtos sem desconto (precision de 0,278 e recall de 0,801). Variantes por tipo de variável (Bernoulli para as d, Gaussiano para as x) poderiam melhorar, mas ficam fora do âmbito pedido.

MLP

A melhor rede tem duas camadas escondidas (64 e 32 neurónios), regularização L2 de 10^{-4}, taxa de aprendizagem de 0,01 e foi treinada com SMOTE. A accuracy cai de 0,864 em CV para 0,789 na validação (−7,5 pontos), com precision de 0,491, o que indica instabilidade do modelo. As redes neuronais exigem em geral mais dados e uma afinação mais fina do que a feita aqui (apenas 24 combinações); a dimensão da amostra (2 940 registos) e a natureza tabular e com muitas variáveis binárias favorecem os modelos de árvores.

Random Forest, Bagging e Extra Trees

Os três usam árvores profundas treinadas em amostras aleatórias, o que reduz a variância face a uma única árvore. A Random Forest e o Bagging beneficiaram do SMOTE (selecionado em ambos). O Bagging obteve a melhor accuracy na validação (0,959, com apenas 8 falsos positivos) e o Extra Trees a melhor AUC (0,977) e a melhor precisão média (0,951) (o que sugere que os cortes aleatórios produzem probabilidades mais bem ordenadas), com o melhor recall (0,861) do grupo juntamente com a árvore de decisão. A Random Forest ficou ligeiramente abaixo (0,950).

AdaBoost e Gradient Boosting

O AdaBoost (300 estimadores, taxa de aprendizagem 1, árvores de profundidade 5, sem reamostragem) obteve a maior accuracy em CV (0,951 ± 0,008), mas na validação ficou em 0,946, com F-measure de 0,861. O Gradient Boosting (200 árvores de profundidade 7, taxa 0,1, subamostragem 0,8) obteve 0,949 em CV e 0,951 na validação, com a precision mais alta do grupo depois do Bagging (0,932). Nenhum dos dois beneficiou do SMOTE; uma possível explicação é que o boosting, ao dar peso crescente aos exemplos mal classificados, é menos sensível ao desequilíbrio, mas esta hipótese não foi testada.

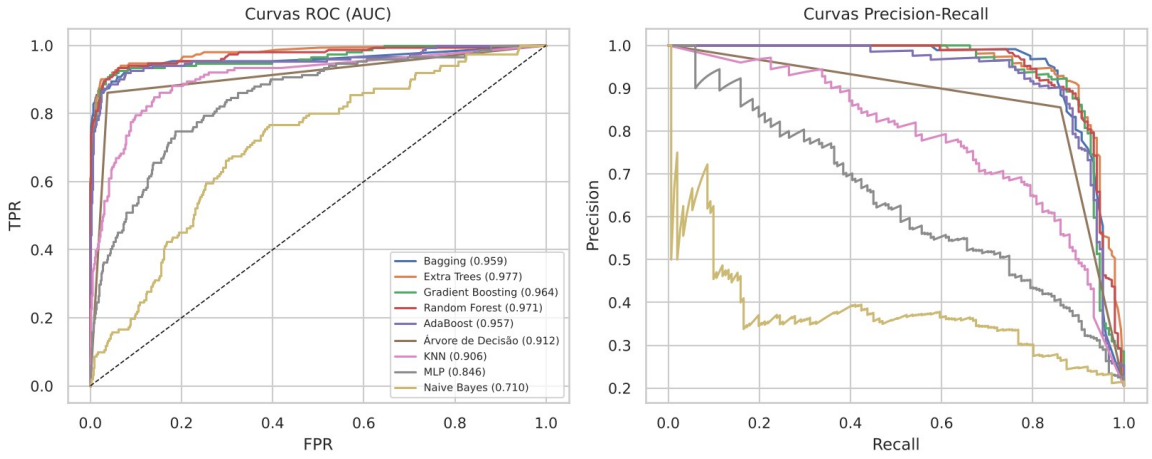


Figura 14 – Curvas ROC (esquerda) e precision-recall (direita) no conjunto de validação.

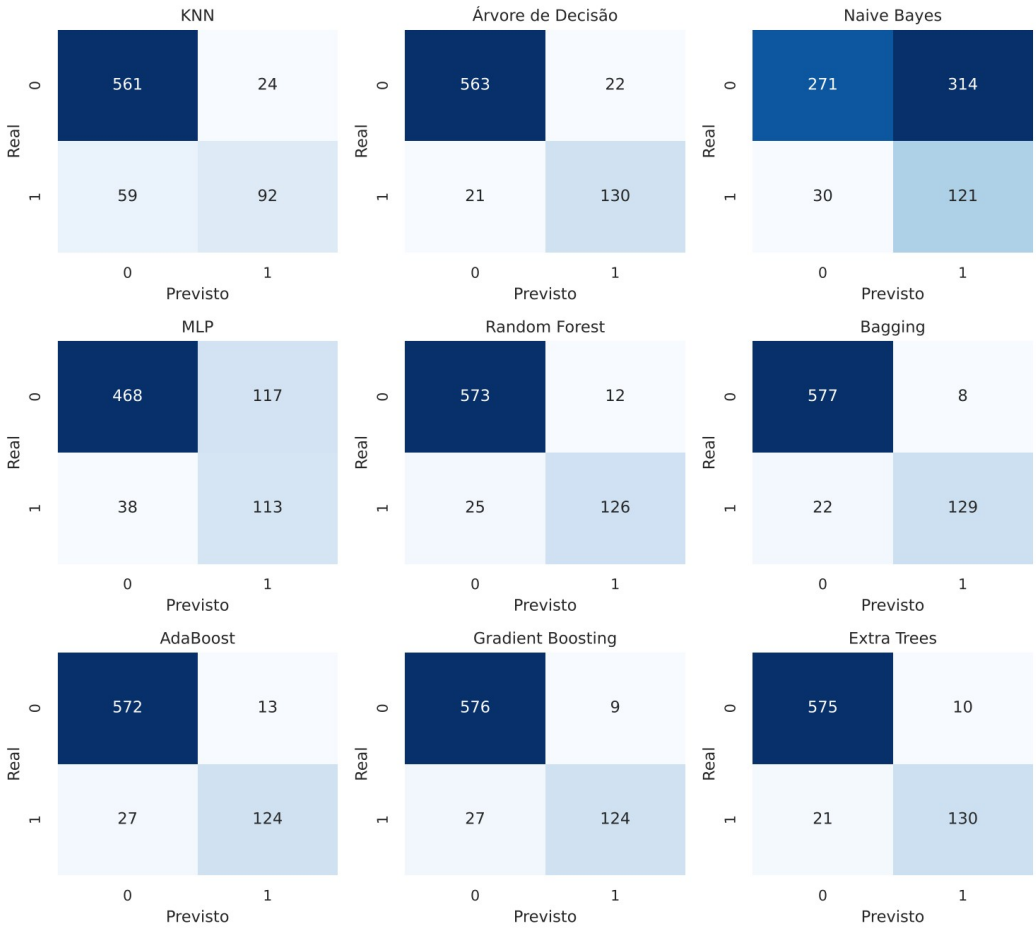


Figura 15 – Matrizes de confusão no conjunto de validação (736 registos: 585 negativos e 151 positivos).

Tabela 13 – Contagens das matrizes de confusão no conjunto de validação (VN = verdadeiros negativos, FP = falsos positivos, FN = falsos negativos, VP = verdadeiros positivos).

Modelo	VN	FP	FN	VP
KNN	561	24	59	92
Árvore de Decisão	563	22	21	130
Naive Bayes	271	314	30	121
MLP	468	117	38	113
Random Forest	573	12	25	126
Bagging	577	8	22	129
AdaBoost	572	13	27	124
Gradient Boosting	576	9	27	124
Extra Trees	575	10	21	130

3.3 Importância das variáveis

As importâncias de Gini do Extra Trees e da Random Forest (Figuras 16 e 17) concordam no essencial: x9 e x5 dominam em ambos os modelos (x9: 0,248 no Extra Trees e 0,207 na Random Forest; x5: 0,142 e 0,176), seguidas das restantes variáveis contínuas (x1, x2, x3, x6, x7 e x8, com valores entre 0,03 e 0,11), e as variáveis discretas com maior importância são d11 e d5, as mesmas que se destacaram na análise exploratória. É notável que x1, x2 e x8, sem associação univariada significativa com y, tenham importância apreciável: isto é compatível com contributos através de interações com

outras variáveis, mas também com um enviesamento conhecido da importância de Gini, que favorece variáveis com mais valores possíveis e é calculada sobre os dados de treino. Deve, por isso, ser lida como indicativa e não como efeito causal. O indicador de x9 em falta tem importância baixa (0,009), já que a informação é em grande parte redundante com a própria x9.

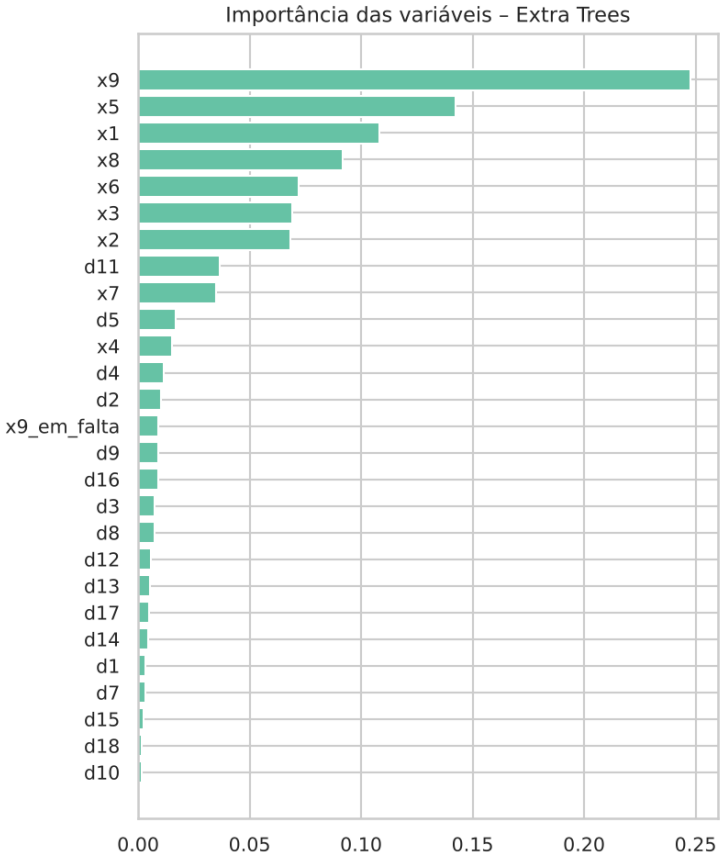


Figura 16 – Importância das variáveis no Extra Trees (ajustado ao treino interno).

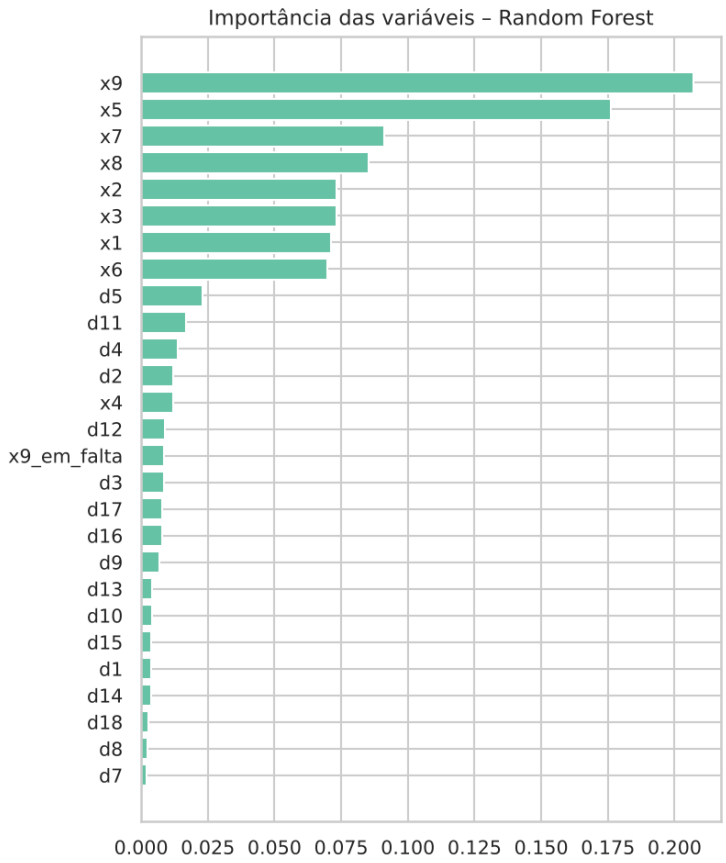


Figura 17 – Importância das variáveis na Random Forest (ajustada ao treino interno).

3.4 Verificações de robustez

Duplicados e fuga de informação. Como existem registos com vetores de características idênticos, uma validação cruzada simples pode colocar cópias do mesmo registo no treino e no teste, o que inflaria a accuracy de modelos que memorizam, como as árvores profundas e o KNN. No treino interno há 101 registos duplicados e 62 dos 736 registos de validação (8,4%) têm um «gémeo» idêntico no treino interno. Para quantificar o efeito repetiu-se a validação cruzada com **StratifiedGroupKFold**, que mantém todos os registos idênticos no mesmo fold. Os resultados (Tabela 14) são semelhantes aos da CV simples (diferenças entre −0,5 e +0,4 pontos): Extra Trees 0,950, Random Forest 0,944, Gradient Boosting 0,944 e KNN 0,873 (CV simples: 0,946, 0,943, 0,949 e 0,870). Não há, portanto, sinais de que os duplicados inflacionem a accuracy estimada. A existência de 382 registos de task.csv (10,4%) com gémeo no treino é uma característica do problema e não um erro: nesses casos os modelos de árvores tendem a reproduzir a etiqueta conhecida, o que beneficia a accuracy na competição.

Tabela 14 – Accuracy em CV simples e em CV agrupada por registos idênticos.

modelo	accuracy CV simples	accuracy CV agrupada
Random Forest	0,9429	0,9439
Extra Trees	0,9463	0,9503
Gradient Boosting	0,9493	0,9439
KNN	0,8704	0,8731

Sobreajuste. A accuracy no treino completo é de 100% para os modelos finais, como é habitual em conjuntos de árvores profundas, e por isso não é indicativa; o que mede a generalização é a CV e a validação, nas quais a diferença entre CV e validação é pequena (≤ 1,8 pontos nos cinco melhores modelos, com a validação geralmente acima da CV).

Ruído nas estimativas. O desvio-padrão da accuracy entre folds (0,5 a 2,7 pontos) e o tamanho do conjunto de validação (736 registos) limitam a resolução da comparação. Para distinguir modelos a menos de um ponto percentual seriam necessárias validações cruzadas repetidas com testes de significância (por exemplo, o teste de McNemar ou o teste t corrigido de Nadeau–Bengio), o que não se fez.

3.5 Modelo final e submissão

Aplicando o critério definido antes de observar a validação – maior accuracy média em validação cruzada no treino interno – o modelo selecionado foi o **AdaBoost** (accuracy em CV = 0,9510; validação: accuracy 0,9457, precision 0,9051, recall 0,8212, F-measure 0,8611).

Este modelo foi reajustado com os 3 676 registos de training.csv e aplicado a task.csv, originando o ficheiro **submission.csv** (colunas i e y, 3 676 linhas e cabeçalho). Foram previstos 20,2% de produtos com desconto, valor coerente com os 20,6% observados no treino. Nenhum dos 30 registos de task.csv com x9 em falta foi classificado como «com desconto», em linha com a regra observada no treino.

Como task.csv não contém o alvo, a accuracy real da submissão é desconhecida; a melhor estimativa é a obtida em validação cruzada e na validação hold-out, isto é, cerca de 0,95 (intervalo plausível de ±0,01–0,02). Dado que os cinco melhores modelos são estatisticamente indistinguíveis, o resultado final na competição poderia ter sido ligeiramente diferente com qualquer um deles.

4. Conclusão

Neste trabalho construiu-se, seguindo o CRISP-DM, um classificador para identificar produtos com desconto. A análise exploratória mostrou que o problema é moderadamente desequilibrado (20,6% de positivos), que as variáveis contínuas são assimétricas e que as correlações lineares com o alvo são fracas, embora 18 das 26 variáveis apresentem associação significativa. Identificou-se que a ausência de x9 é informativa (todos os 49 casos têm y = 0). A preparação dos dados incluiu a divisão em treino interno e validação, imputação com indicador de ausência, estudo de outliers (não removidos), de estratégias de reamostragem (tratadas como hiperparâmetro) e da redundância de variáveis (todas mantidas).

Dos nove algoritmos comparados, os conjuntos de árvores (Bagging, Extra Trees, Gradient Boosting, Random Forest e AdaBoost) alcançaram accuracy entre 94,6% e 95,9% na validação, F-measure entre 0,86 e 0,90 e AUC entre 0,96 e 0,98, bastante acima da baseline de 79,4%, do KNN (88,7%), do MLP (78,9%) e do Naive Bayes (53,3%). A superioridade das árvores é coerente com a natureza dos dados: muitas variáveis binárias, variáveis de baixa cardinalidade, assimetrias e interações. Os cinco melhores modelos são, no entanto, estatisticamente indistinguíveis; o AdaBoost foi escolhido por apresentar a maior accuracy em validação cruzada.

Limitações

- O significado das variáveis é desconhecido, pelo que as interpretações (por exemplo, x5 como preço ou as variáveis d como codificação de categorias) são hipóteses.
- A métrica da competição é a accuracy, que dá o mesmo peso aos dois tipos de erro; num contexto de negócio, os custos de falsos positivos e falsos negativos podem diferir e justificar a otimização do recall ou da F-measure (ou a escolha de um limiar de decisão diferente de 0,5).
- As grelhas de hiperparâmetros são limitadas (sobretudo para o MLP) e não se aplicaram testes de significância entre modelos.
- Os registos duplicados e a sobreposição entre treino e teste (10,4% de task.csv com gémeo no treino) beneficiam modelos que memorizam; o desempenho em produtos novos poderá ser inferior ao estimado.

Trabalho futuro

- Validação cruzada repetida e testes estatísticos (McNemar, Nadeau–Bengio) para comparar os melhores modelos.
- Ajuste do limiar de decisão e calibração das probabilidades, para otimizar a métrica pretendida.
- Modelos adicionais (XGBoost, LightGBM, empilhamento/stacking e votação) e uma afinação mais extensa do MLP.
- Implementação (fase final do CRISP-DM): disponibilizar o modelo, monitorizar a accuracy em novos dados e reavaliar periodicamente, dado que as políticas de desconto mudam ao longo do tempo (deriva de conceito).

Referências

Breiman, L. (1996). Bagging predictors. Machine Learning, 24(2), 123–140.

Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32.

Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). Classification and Regression Trees. Wadsworth.

Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). CRISP-DM 1.0: Step-by-step data mining guide. SPSS Inc.

Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. Journal of Artificial Intelligence Research, 16, 321–357.

Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. IEEE Transactions on Information Theory, 13(1), 21–27.

Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. Journal of Computer and System Sciences, 55(1), 119–139.

Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. Annals of Statistics, 29(5), 1189–1232.

Geurts, P., Ernst, D., & Wehenkel, L. (2006). Extremely randomized trees. Machine Learning, 63(1), 3–42.

Lemaître, G., Nogueira, F., & Aridas, C. K. (2017). Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. Journal of Machine Learning Research, 18(17), 1–5.

Pedregosa, F., et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830.

Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. Nature, 323, 533–536.

Anexo A – Hiperparâmetros e grelhas de pesquisa

Tabela 15 – Grelhas de pesquisa (5 folds por combinação; critério: accuracy).

algoritmo	grelha	combinações
KNN	n_neighbors ∈ {3,5,7,11,15,25}; weights ∈ {uniform, distance}; p ∈ {1,2}; reamostragem ∈ {nenhuma, SMOTE}	48
Árvore de Decisão	max_depth ∈ {4,6,8,12,None}; min_samples_leaf ∈ {1,5,10}; criterion ∈ {gini, entropy}; reamostragem	60
Naive Bayes	var_smoothing ∈ {1e-9,1e-6,1e-3,1e-1}; reamostragem	8
MLP	hidden_layer_sizes ∈ {(32),(64),(64,32)}; alpha ∈ {1e-4,1e-2}; learning_rate_init ∈ {1e-3,1e-2}; reamostragem; early stopping, máx. 600 iterações	24
Random Forest	n_estimators ∈ {300,600}; max_features ∈ {sqrt, 0,5}; min_samples_leaf ∈ {1,2}; reamostragem	16
Bagging	n_estimators ∈ {50,200}; max_samples ∈ {0,7; 1,0}; profundidade da árvore base ∈ {None,12}; reamostragem	16
AdaBoost	n_estimators ∈ {100,300}; learning_rate ∈ {0,3; 1,0}; profundidade da árvore base ∈ {1,3,5}; reamostragem	24
Gradient Boosting	n_estimators ∈ {200,500}; learning_rate ∈ {0,05; 0,1}; max_depth ∈ {3,5,7}; subsample ∈ {0,8; 1,0}; reamostragem (pesquisa aleatória)	12 de 48
Extra Trees	n_estimators ∈ {300,600}; max_features ∈ {sqrt, 0,5, 0,8}; min_samples_leaf ∈ {1,2}; reamostragem	24

Tabela 16 – Hiperparâmetros selecionados por validação cruzada.

algoritmo	hiperparâmetros selecionados	reamostragem
KNN	n_neighbors=7; p=1; weights=distance	nenhuma
Árvore de Decisão	criterion=gini; max_depth=None; min_samples_leaf=1	SMOTE
Naive Bayes	var_smoothing=0.1	nenhuma
MLP	alpha=0.0001; hidden_layer_sizes=[64, 32]; learning_rate_init=0.01	SMOTE
Random Forest	max_features=0.5; min_samples_leaf=1; n_estimators=600	SMOTE
Bagging	base_max_depth=None; max_samples=1.0; n_estimators=50	SMOTE
AdaBoost	base_max_depth=5; learning_rate=1.0; n_estimators=300	nenhuma
Gradient Boosting	subsample=0.8; n_estimators=200; max_depth=7; learning_rate=0.1	nenhuma
Extra Trees	max_features=0.8; min_samples_leaf=1; n_estimators=600	nenhuma

Anexo B – Estrutura do código e reprodutibilidade

Todo o código encontra-se na pasta project/ e pode ser executado pela ordem seguinte, em Python 3 com as bibliotecas pandas, numpy, scipy, statsmodels, scikit-learn, imbalanced-learn, matplotlib, seaborn e python-docx:

- **common.py** – leitura dos dados, divisão treino/validação (80/20 estratificada), objeto de validação cruzada (5 folds) e transformador Winsorizer.
- **eda.py** – análise exploratória: estatísticas descritivas, histogramas, diagramas de caixa, correlações e testes de associação.
- **prep.py** – preparação dos dados: valores em falta, outliers, desequilíbrio de classes, redundância e composição dos folds.
- **model.py** – afinação, avaliação e comparação dos nove modelos; importâncias e verificações de robustez.
- **final.py** – seleção do modelo, reajuste com todo o treino e geração de submission.csv.
- **replot.py** e **build_report.py** – versão final de duas figuras e geração deste relatório.
- **results/** – tabelas em CSV/JSON; **figs/** – figuras; **data/** – dados e enunciado.